PL/SQL Seminar Exercises: Comprehensive Practice Set

Category 1: PL/SQL Fundamentals & Block Structure

Beginner Level (5 exercises)

1. Create a PL/SQL block that declares variables for an employee's ID, first name, and last name. Initialize these variables with values for employee ID 100 from the employees table. Display the employee's full name using DBMS*OUTPUT.PUT*LINE.
2. Create a PL/SQL block that updates the salary for employee ID 100 by adding 100. After the update, verify if the update was successful using SQL%ROWCOUNT and display an appropriate message.
3. Create a PL/SQL block that prompts for an employee ID using a substitution variable and displays the employee's name. Handle the case where the employee ID doesn't exist by displaying an appropriate message.
4. Create a PL/SQL block that calculates the average salary for all employees and displays the result. Use appropriate variable declarations matching the data types in the employees table.
5. Create a PL/SQL block that declares a constant for the minimum salary (1000.0) and displays this value. Show how constants cannot be modified by attempting to change this value within the block.

Intermediate Level (5 exercises)

1. Create a PL/SQL block that retrieves employee information (first*name, last*name, hire_date) for employee ID 100 using scalar variables. Format and display the hire date in 'DD-MON-YYYY' format.
2. Create a PL/SQL block that uses bind variables to count the number of employees in department ID 30. Execute the block using VARIABLE and PRINT commands, then verify the result.
3. Create a PL/SQL block that calculates the difference between the highest and lowest salary in the employees table. Store this difference in a variable and display the result with an appropriate message.
4. Create a PL/SQL block that prompts for a department ID and displays the department name by joining employees and departments tables. Handle cases where the department doesn't exist.
5. Create a PL/SQL block that updates the hire date for all employees in department ID 30 by adding 7 days. Display the number of employees updated using SQL%ROWCOUNT.

Difficulty Level (5 exercises)

1. Create a PL/SQL block that calculates the percentage of employees with salary above 7000. Use multiple scalar variables to store intermediate results and display the final percentage with appropriate formatting.
2. Create a PL/SQL block that compares the average salary between two departments (ID 30 and 60). Display which department has the higher average salary or if they are equal. Handle cases where one or both departments have no employees.
3. Create a PL/SQL block that uses context switching analysis to compare the performance of two approaches: one using multiple individual SELECT statements and one using a single SELECT with multiple aggregates.
4. Create a PL/SQL block that calculates the employment growth rate (percentage increase) from the previous year to the current year. Use appropriate date functions and handle cases where there's no data for the previous year.
5. Create a PL/SQL block that demonstrates the difference between substitution variables and bind variables by creating two versions of the same functionality that retrieves employee information based on an ID.

Hard Level (5 exercises)

1. Create a PL/SQL block that analyzes the memory usage of different variable declaration approaches when processing employee data. Compare %TYPE vs explicit declarations, scalar vs record types, and demonstrate the performance implications.

2. Create a PL/SQL block that implements a complex employee eligibility check using multiple conditions across different tables. The block should verify job title, department, and salary range before approving eligibility.
3. Create a PL/SQL block that demonstrates the impact of context switching on performance by processing employee data in three different ways: row-by-row, bulk collect without limit, and bulk collect with limit, comparing execution statistics.
4. Create a PL/SQL block that implements a commission validation system, checking if employees have met sales targets for commission eligibility, using multiple joins and conditional logic.
5. Create a PL/SQL block that analyzes the performance characteristics of different variable declaration patterns when processing large datasets from the employees and orders tables, with detailed commentary on optimization opportunities.

Category 2: Variables, Constants & Data Types

Beginner Level (5 exercises)

1. Create a PL/SQL block that declares variables using %TYPE for employee*id, first*name, and last_name from the employees table. Retrieve and display information for employee ID 100.
2. Create a PL/SQL block that uses %ROWTYPE to retrieve and display all columns for department ID 30 from the departments table.
3. Create a PL/SQL block that declares a constant for the maximum salary (500000.0) and displays this value. Attempt to modify the constant to demonstrate it cannot be changed.
4. Create a PL/SQL block that uses a substitution variable to prompt for an employee ID and displays the employee's first and last name using %TYPE declarations.
5. Create a PL/SQL block that declares a user-defined record type to store employee information (employee*id, first*name, last_name) and uses it to retrieve employee ID 100's information.

Intermediate Level (5 exercises)

1. Create a PL/SQL block that uses an index-by table (associative array) to store department names as keys and employee counts as values. Populate this collection by querying the database and display all entries.
2. Create a PL/SQL block that uses a nested table to store multiple order values for a single client. Initialize the nested table, add several order values, and calculate the average order value.
3. Create a PL/SQL block that compares the usage of %TYPE vs explicit data type declarations by implementing the same functionality two different ways and discussing the advantages of %TYPE.
4. Create a PL/SQL block that uses a user-defined record to combine data from employees and departments tables for employee ID 100 and displays the combined information.
5. Create a PL/SQL block that demonstrates the difference between NOT NULL variables with and without initialization by attempting to declare and use such variables.

Difficulty Level (5 exercises)

1. Create a PL/SQL block that implements a department capacity monitoring system using nested tables to track employee counts across multiple time periods. Calculate staffing trends and display the results.
2. Create a PL/SQL block that uses associative arrays with VARCHAR2 keys to implement an employee commission lookup system by job title, demonstrating efficient in-memory data access.
3. Create a PL/SQL block that compares the memory usage and performance of different collection types (nested tables, varrays, associative arrays) when processing employee data.
4. Create a PL/SQL block that implements a dynamic employee classification system using user-defined records with calculated fields, showing how complex data structures can simplify business logic.
5. Create a PL/SQL block that demonstrates the proper initialization and usage of nested tables in DML operations, including inserting, updating, and querying nested table data.

Hard Level (5 exercises)

1. Create a PL/SQL block that implements an advanced employee performance analytics system using multiple collection types to track salaries, commissions, and performance metrics, with comprehensive reporting capabilities.
2. Create a PL/SQL block that analyzes the memory footprint and processing efficiency of different variable declaration strategies when handling large employee datasets, with detailed performance metrics and recommendations.
3. Create a PL/SQL block that implements a complex organizational structure using hierarchical data structures to track manager-employee relationships and department hierarchies.
4. Create a PL/SQL block that demonstrates the integration of different collection types (nested tables, varrays, associative arrays) in a single workflow for processing employee data with detailed commentary on appropriate usage scenarios.
5. Create a PL/SQL block that implements a real-time employee monitoring system using advanced data structures to track performance metrics, with dynamic threshold adjustments based on historical data.

Category 3: Control Structures & Program Flow

Beginner Level (5 exercises)

1. Create a PL/SQL block that retrieves an employee's salary (ID 100) and uses an IF statement to display "Senior" if salary >= 7000, "Mid-level" if between 3000-6999, and "Junior" otherwise.
2. Create a PL/SQL block that uses a simple LOOP to display numbers 1 through 10. Include an EXIT WHEN condition to terminate the loop.
3. Create a PL/SQL block that uses a WHILE loop to process employee IDs from 100 to 110, displaying each employee's first name. Handle cases where employees don't exist.
4. Create a PL/SQL block that uses a FOR loop with numeric range to display department IDs 10 through 60, and their corresponding names.
5. Create a PL/SQL block that uses a CASE statement to convert a numeric salary (5000) to a job category ('Senior', 'Mid-level', 'Junior') and display the result.

Intermediate Level (5 exercises)

1. Create a PL/SQL block that uses a CASE expression to determine employee category (Senior, Mid-level, Junior) based on salary and displays the result for employee ID 100.
2. Create a PL/SQL block that uses nested loops to display the department-employee hierarchy: for each department, list all employees, and for each employee, display their salary.
3. Create a PL/SQL block that processes employee hire dates using a WHILE loop, calculating how many employees were hired in each quarter of the current year.
4. Create a PL/SQL block that uses a cursor FOR loop to display the top 5 employees with the highest salary in department ID 30, ordered by salary descending.
5. Create a PL/SQL block that implements a salary validation system using multiple IF-ELSIF conditions to verify salary values are within acceptable ranges for job titles.

Difficulty Level (5 exercises)

1. Create a PL/SQL block that implements a comprehensive employee eligibility workflow using multiple control structures to validate job title, department, and salary range.
2. Create a PL/SQL block that compares the performance of different loop structures (simple LOOP, WHILE, FOR) when processing employee data, with detailed execution statistics and commentary.
3. Create a PL/SQL block that uses CASE statements to implement a complex commission calculation system that calculates commissions based on salary, job title, and sales performance.
4. Create a PL/SQL block that processes employee data using nested cursor FOR loops to display department → job → employee hierarchy with salary information at each level.

5. Create a PL/SQL block that implements an employee retention prediction model using multiple conditional structures to analyze salary, tenure, and job title.

Hard Level (5 exercises)

1. Create a PL/SQL block that implements an advanced HR workflow engine using multiple control structures to manage employee hiring, promotions, and salary adjustments with comprehensive business rule validation.
2. Create a PL/SQL block that analyzes the performance characteristics of different control structure implementations when processing large employee datasets, with detailed metrics and optimization recommendations.
3. Create a PL/SQL block that implements a dynamic job recommendation system using complex conditional logic to analyze employee salary, tenure, and job title.
4. Create a PL/SQL block that demonstrates the integration of multiple control structures in a single workflow for processing employee data with detailed commentary on appropriate usage scenarios and performance considerations.
5. Create a PL/SQL block that implements a real-time HR intervention system using advanced control structures to identify at-risk employees based on multiple performance indicators and trigger appropriate interventions.

Category 4: Cursors & Record Processing

Beginner Level (5 exercises)

1. Create a PL/SQL block that declares an explicit cursor to select employee*id, first*name, and last*name from employees where department*id = 30. Process each row and display the information.
2. Create a PL/SQL block that uses an implicit cursor to update the salary for employee ID 100 and verify the update using SQL%ROWCOUNT.
3. Create a PL/SQL block that declares a cursor to retrieve department information and uses cursor attributes (%FOUND, %NOTFOUND, %ROWCOUNT) to control processing and display statistics.
4. Create a PL/SQL block that uses a parameterized cursor to retrieve employees for a specific department ID (use department ID 30) and display the results.
5. Create a PL/SQL block that uses a cursor FOR loop to display the first 5 employees with the highest salary in department ID 30.

Intermediate Level (5 exercises)

1. Create a PL/SQL block that uses a cursor to retrieve and process employees with salary below 3000, updating their status to "Below Minimum" and displaying the number of employees updated.
2. Create a PL/SQL block that implements a nested cursor structure to display the department → job → employee hierarchy, showing the number of employees at each level.
3. Create a PL/SQL block that uses a cursor FOR UPDATE to adjust salaries for employees in department ID 30, increasing by 10%, and commits the changes.
4. Create a PL/SQL block that uses BULK COLLECT to retrieve all employees in department ID 30 into a collection and process them in memory, displaying statistics about the operation.
5. Create a PL/SQL block that implements a parameterized cursor to find employees based on salary range (e.g., 5000-10000) and display the results with appropriate formatting.

Difficulty Level (5 exercises)

1. Create a PL/SQL block that implements a comprehensive employee analysis using multiple cursors to process department, job, and employee data with hierarchical relationships.
2. Create a PL/SQL block that compares the performance of row-by-row cursor processing versus BULK COLLECT for large employee datasets, with detailed execution statistics and commentary.

3. Create a PL/SQL block that implements a dynamic employee reporting system using REF CURSOR to return different result sets based on runtime parameters.
4. Create a PL/SQL block that processes employee data using cursor variables with proper resource management, including opening, fetching, and closing operations.
5. Create a PL/SQL block that implements an employee retention analysis system using nested cursors to track tenure and salary progression across time.

Hard Level (5 exercises)

1. Create a PL/SQL block that implements an advanced HR analytics engine using multiple cursor techniques to analyze employee performance, salary progression, and organizational structure across the company.
2. Create a PL/SQL block that demonstrates the integration of different cursor processing techniques (explicit, implicit, BULK COLLECT, REF CURSOR) in a single workflow for comprehensive employee data analysis.
3. Create a PL/SQL block that implements a real-time employee monitoring system using advanced cursor techniques to track performance metrics with dynamic filtering and aggregation.
4. Create a PL/SQL block that analyzes the performance characteristics of different cursor processing approaches when handling large employee datasets, with detailed metrics and optimization recommendations.
5. Create a PL/SQL block that implements a predictive analytics system for employee success using advanced cursor processing to analyze historical performance data and identify at-risk patterns.

Category 5: Error Handling & Exception Management

Beginner Level (5 exercises)

1. Create a PL/SQL block that retrieves an employee's information (ID 100) and handles the NO*DATA*FOUND exception if the employee doesn't exist.
2. Create a PL/SQL block that attempts to divide by zero and handles the ZERO_DIVIDE exception with an appropriate message.
3. Create a PL/SQL block that updates an employee's salary to a negative value and handles the resulting exception using SQLCODE and SQLERRM.
4. Create a PL/SQL block that uses a WHEN OTHERS clause to catch any unexpected errors during employee data processing, logging the error details.
5. Create a PL/SQL block that validates an employee's hire date and uses RAISE*APPLICATION*ERROR to prevent future dates, with appropriate error message.

Intermediate Level (5 exercises)

1. Create a PL/SQL block that implements multiple exception handlers for employee data processing, including specific handlers for NO*DATA*FOUND, TOO*MANY*ROWS, and a general WHEN OTHERS clause.
2. Create a PL/SQL block that uses user-defined exceptions with PRAGMA EXCEPTION_INIT to handle specific Oracle error codes during employee updates.
3. Create a PL/SQL block that implements nested exception handling blocks to process employee data with different validation rules at each level.
4. Create a PL/SQL block that logs error details to an error_log table during employee data processing, including SQLCODE, SQLERRM, and contextual information.
5. Create a PL/SQL block that demonstrates exception propagation across nested PL/SQL blocks during complex employee processing.

Difficulty Level (5 exercises)

1. Create a PL/SQL block that implements a comprehensive employee validation system with custom business rule exceptions and detailed error logging.

2. Create a PL/SQL block that handles transactional integrity during complex employee updates, using savepoints and proper exception handling.
1. Create a PL/SQL block that implements a robust error handling framework for HR operations with detailed context-specific error messages and recovery procedures.
2. Create a PL/SQL block that analyzes the performance impact of different exception handling strategies during high-volume employee data processing operations.
3. Create a PL/SQL block that implements a dynamic error handling system that adapts its response based on the type and severity of errors encountered during HR processing.

Hard Level (5 exercises)

1. Create a PL/SQL block that implements an enterprise-grade error management system for HR operations with comprehensive logging, notification, and recovery capabilities.
2. Create a PL/SQL block that demonstrates the integration of advanced error handling techniques in a complex HR workflow with multiple transaction boundaries and recovery points.
3. Create a PL/SQL block that implements a predictive error prevention system that identifies potential issues before they occur during employee data processing.
4. Create a PL/SQL block that analyzes and optimizes exception handling patterns across multiple HR processing workflows, with detailed performance metrics and recommendations.
5. Create a PL/SQL block that implements a real-time error monitoring and response system for HR operations with dynamic adjustment of error handling strategies based on system conditions.

Category 6: Stored Procedures & Functions

Beginner Level (5 exercises)

1. Create a procedure get*employee*info that takes an employee*id as input and displays the employee's first*name, last*name, and department*name using DBMS_OUTPUT.
2. Create a function get*employee*count that takes a department_id as input and returns the number of employees in that department. Call the function from a SELECT statement.
3. Create a procedure update*employee*salary that takes employee*id and new*salary as parameters and updates the employee's salary. Include proper exception handling.
4. Create a function calculate*job*category that takes a numeric salary as input and returns the corresponding job category ('Senior', 'Mid-level', 'Junior'). Test the function with various salary values.
5. Create a procedure get*top*employees that takes a department_id and limit as parameters and displays the top employees by salary (up to the specified limit).

Intermediate Level (5 exercises)

1. Create a package employee_pkg that includes:
    - A function to get employee count by department
    - A function to get average salary by department
    - A procedure to display comprehensive department statistics

    Invoke the package procedures and functions.
2. Create a procedure process*salary*changes that uses an autonomous transaction to log salary changes while updating employee records, demonstrating transaction independence.
3. Create a function is*senior*employee that takes employee_id as input and returns 'YES' if salary >= 7000, otherwise 'NO'. Ensure the function can be used in SQL statements.
4. Create a procedure transfer_employee that moves an employee between departments, updating relevant records and implementing proper transaction control with exception handling.

5. Create a function get*employee*performance that takes employee_id as input and returns a formatted string with salary, job category, and performance metrics.

Difficulty Level (5 exercises)

1. Create a package hr_pkg that includes:
    - A procedure to validate employee eligibility
    - A procedure to process job changes
    - A function to calculate average salary

    Implement proper exception handling and transaction control throughout the package.
2. Create a procedure batch*update*salaries that processes salary updates in batches of 100 employees, with progress reporting and error handling for individual record failures.
3. Create a function get*department*hierarchy that returns a REF CURSOR with the complete department-job-employee hierarchy, demonstrating dynamic result set capabilities.
4. Create a procedure analyze*employee*trends that implements complex HR trend analysis using multiple data points and returns comprehensive metrics through OUT parameters.
5. Create a function calculate*employee*value that calculates an employee's organizational value based on multiple factors, demonstrating complex business logic encapsulation.

Hard Level (5 exercises)

1. Create a comprehensive hr*services*pkg package that implements a full suite of employee management functionality with proper security, transaction control, and error handling.
2. Create a procedure optimize*hr*workflow that implements an advanced batch processing system for employee operations with dynamic resource allocation and performance monitoring.
3. Create a function predict*employee*success that uses historical data and multiple metrics to predict employee success probabilities, demonstrating complex analytical capabilities.
4. Create a procedure implement*hr*policy that enforces complex company policies across multiple data points with comprehensive validation and exception handling.
5. Create a package analytics*engine*pkg that provides advanced analytical capabilities for employee performance, department trends, and resource utilization with optimized performance characteristics.

Category 7: Advanced PL/SQL Features

Beginner Level (5 exercises)

1. Create a trigger trg*check*salary that prevents inserting or updating employee records with salary outside the 1000.0-500000.0 range, using RAISE*APPLICATION*ERROR for invalid values.
2. Create a trigger trg*salary*audit that logs changes to the employees table to an audit table whenever INSERT, UPDATE, or DELETE operations occur.
3. Create a procedure dynamic_query that takes a table name and employee ID as parameters and uses EXECUTE IMMEDIATE to retrieve and display employee information.
4. Create a function get*department*employees that returns a SYS_REFCURSOR with employee details for a given department ID, demonstrating dynamic result set capabilities.
5. Create a PL/SQL block that uses hierarchical queries to display the department-manager structure using START WITH and CONNECT BY clauses.

Intermediate Level (5 exercises)

1. Create a trigger trg*department*capacity that prevents reducing department capacity below current employee count, using :OLD and :NEW qualifiers appropriately.

2. Create a procedure archive_employees that uses dynamic SQL to archive and delete historical employee data older than 10 years, maintaining referential integrity.
3. Create a function get*employee*hierarchy that returns a hierarchical representation of employee data using SYS*CONNECT*BY_PATH and LEVEL.
4. Create a procedure process*employee*batch that implements batch processing of employee records using collections and BULK COLLECT for performance optimization.
5. Create a recursive function get*total*employees that calculates total employees for a department including all sub-departments in a hierarchical structure.

Difficulty Level (5 exercises)

1. Create a comprehensive hr*rules*pkg package that implements business rules through a combination of triggers, procedures, and functions with proper error handling.
2. Create a dynamic*reporting*engine procedure that generates custom reports based on user-specified parameters using advanced dynamic SQL techniques with proper security measures.
3. Create a hierarchical*analysis*pkg package that provides advanced capabilities for analyzing organizational hierarchies (department → job → employee) with performance-optimized queries.
4. Create a procedure optimize*hr*workflow that implements an advanced processing system for HR operations with dynamic resource allocation and performance monitoring.
5. Create a trigger system that implements comprehensive audit logging for all HR data changes with historical tracking and change analysis capabilities.

Hard Level (5 exercises)

1. Create an enterprise*hr*framework package that integrates triggers, dynamic SQL, hierarchical queries, and collections to implement a comprehensive HR management system.
2. Create a predictive*analytics*engine procedure that uses advanced PL/SQL features to analyze historical HR data and predict future trends and outcomes.
3. Create a real*time*monitoring_system that implements continuous monitoring of HR operations using advanced PL/SQL features with dynamic alerting and response capabilities.
4. Create a performance*optimization*framework that analyzes and optimizes PL/SQL code across the HR database, providing detailed recommendations and implementation guidance.
5. Create a security*enforcement*system that implements comprehensive security controls across all HR operations using advanced PL/SQL features with detailed auditing and compliance reporting.